

Problem 16 Give an Algorithm for finding the Number of half nodes (nodes with only one child) in the Binary Tree without Using Recursion

Solution: The Set of all nodes with either left or right child (but not both) are called half nodes.

Algorithm

1. if root == null, return 0.
2. Create an empty Queue Q.
3. Enqueue root into Q
4. Initialize halfCount = 0
5. While Q is not Empty
 - Dequeue a node curr
 - if Exactly one of curr.left or curr.right is not null
 - increment halfCount
 - if curr.left exists, Enqueue it
 - if curr.right exists, Enqueue it
6. Return halfCount

code

```
public static int countHalfNodes(TreeNode root) {
    if (root == null)
        return 0;

    Queue<TreeNode> q = new LinkedList<>();
    q.add(root);

    int halfCount = 0;

    while (!q.isEmpty()) {
        TreeNode cur = q.poll();

        if ((cur.left != null && cur.right == null) ||
            (cur.left == null && cur.right != null)) {
            halfCount++;
        }

        if (cur.left != null)
            q.add(cur.left);
        if (cur.right != null)
            q.add(cur.right);
    }

    return halfCount;
}
```

Problem 17 Given two Binary trees, Returns true if they are Structurally identical

Solution : Two Binary trees are identical if

1. They have the same structure
2. Corresponding nodes having the same data
3. Left & Right Subtrees are also identical

Algorithm

1. if Both root 1 & root 2 are null return true
2. if one is null other is not return false
3. if root 1.data $\neq$ root 2.data, return false
4. Recursively check
 - left Subtree
 - Right Subtree
5. Return true if Both Subtrees are identical

Code

```
public class IdenticalTrees {  
    public static boolean areIdentical(TreeNode root1, TreeNode root2) {  
        // both empty  
        if (root1 == null && root2 == null)  
            return true;  
  
        // one empty, one not  
        if (root1 == null || root2 == null)  
            return false;  
  
        // data mismatch  
        if (root1.data != root2.data)  
            return false;  
  
        // check left and right subtrees  
        return areIdentical(root1.left, root2.left) &&  
            areIdentical(root1.right, root2.right);  
    }  
}
```

Problem 18 Give an algorithm for finding a diameter (depth) of a tree

Diameter (depth) of a Binary tree is a longest path between two leaves in a Binary Tree.

Solution: To find the diameter of a tree, first calculate the diameter of left subtree and right subtree recursively. Among these two values we need to send maximum value along with current level (+1).

Algorithm

1. Initialize a static variable `diameter = 0`
2. Define a Recursive function `height(node)`
 - if `node == null`, return 0
 - Compute height of left subtree
 - Compute height of Right subtree
 - Update
$$\text{diameter} = \max(\text{diameter}, \text{leftHeight} + \text{rightHeight} + 1)$$
 - Return $1 + \max(\text{leftHeight}, \text{rightHeight})$
 - Call `height(root)`
 - Return `diameter`

Code

```
public class DiameterOfTree {  
  
    static int diameter = 0;  
  
    public static int height(TreeNode root) {  
        if (root == null)  
            return 0;  
  
        int leftHeight = height(root.left);  
        int rightHeight = height(root.right);  
  
        // update diameter  
        diameter = Math.max(diameter, leftHeight + rightHeight + 1);  
  
        return 1 + Math.max(leftHeight, rightHeight);  
    }  
  
    public static int findDiameter(TreeNode root) {  
        diameter = 0;  
        height(root);  
        return diameter;  
    }  
}
```